MenteIng

Challenge 5 Funciones y Reportes

Funciones que retornan Tabla de Registros Función 1: ReporteAlertasPorCarrera(p_fecha_inicio, p_fecha_fin)

El decanato no tenía forma de saber qué carreras están concentrando más alertas de riesgo en un período dado.

Sin eso, no podés hacer intervenciones institucionales a tiempo.

La función recibe un rango de fechas y retorna una tabla con los siguientes datos por carrera:

- Nombre de la carrera.
- Cantidad de estudiantes distintos con alertas en el periodo.
- Tipo de alerta más frecuente.
- Total de alertas generadas. **Parámetros de entrada:**

Parámetro	Tipo	Descripción
p_fecha_inicio	DATE	Fecha de inicio del periodo
p_fecha_fin	DATE	Fecha de fin del periodo

Implementación SQL:

```
CREATE OR REPLACE FUNCTION ReporteAlertasPorCarrera( p_fecha_inicio DATE, p_fecha_fin DATE ) RETURNS TABLE ( NombreCarrera VARCHAR, TotalEstudiantes BIGINT, TipoAlertaFrecuente VARCHAR, TotalAlertas BIGINT ) LANGUAGE plpgsql

AS $$ BEGIN RETURN QUERY SELECT c.NombreCarrera::VARCHAR, COUNT(DISTINCT e.Matricula), MODE() WITHIN

GROUP (ORDER BY a.TipoAlerta)::VARCHAR, COUNT(a.IdAlerta) FROM Alerta a JOIN Estudiante e ON a.Matricula =
e.Matricula JOIN Carrera c ON e.IdCarrera = c.IdCarrera WHERE a.FechaAlerta BETWEEN p_fecha_inicio AND p_fecha_fin GROUP BY c.NombreCarrera ORDER BY TotalAlertas DESC; END; $$; -Uso: SELECT * FROM ReporteAlertasPorCarrera('202501-01', '2025-06-30');
```

Resultado esperado:

NombreCarrera	TotalEstudiantes	TipoAlertaFrecuente	TotalAlertas
Ingeniería en Sistemas	4	Estrés Alto	9
Ingeniería Civil	2	Sueno Bajo	4
Ingeniería Electrónica	1	Riesgo Burnout	2
Ingeniería Industrial	1	Estrés Alto	2

Función 2: HistorialBienestarEstudiante(p_matricula)

Como el profesional no tiene una vista unificada del historial de bienestar de un paciente antes de la cita, debiendo consultar múltiples tablas manualmente.

La función recibe la matrícula de un estudiante y retorna su historial completo combinando:

- Nivel de estrés diario.

FechaRegistro	NivelEstres	RangoHoras	EtiquetaAnimo	NivelAnimo

- Rango y tipo de horas de sueño.
- Etiqueta y nivel numérico del estado de ánimo.
- Comentarios libres del estudiante.
- Ordenado cronológicamente de más reciente a más antiguo.

Parámetros de entrada:

Parámetro	Tipo	Descripción
p_matricula	VARCHAR(20)	Matricula del estudiante a consultar

Implementación SQL:

```
CREATE OR REPLACE FUNCTION HistorialBienestarEstudiante( p_matricula
VARCHAR(20) ) RETURNS
TABLE ( FechaRegistro TIMESTAMP, NivelEstres INT, RangoHoras VARCHAR,
TipoSueno VARCHAR,
EtiquetaAnimo VARCHAR, NivelAnimo INT, Comentario TEXT ) LANGUAGE plpgsql
AS $$ BEGIN RETURN QUERY SELECT rd.FechaRegistro, rd.NivelEstres,
hs.RangoHoras::VARCHAR, hs.TipoSueno::VARCHAR, ea.EtiquetaAnimo::VARCHAR,
ea.NivelAnimo, ea.Comentario FROM RegistroDiario rd LEFT JOIN HorasSueno hs ON
hs.IdRegistro = rd.IdRegistro LEFT JOIN EstadoAnimo ea ON ea.IdRegistro =
rd.IdRegistro WHERE rd.Matricula = p_matricula ORDER BY rd.FechaRegistro DESC;
END; $$; -- Uso: SELECT * FROM HistorialBienestarEstudiante('2021-0001');
```

Resultado esperado:

2025-05-10 21:00:00	9	menos de 6 h	Ansioso	7
2025-05-09 22:00:00	8	menos de 6 h	Agotado	6
2025-05-08 20:30:00	7	8 horas en prom.	Neutral	6
2025-05-07 21:15:00	8	menos de 6 h	Ansioso	7

Función 3: PromedioEstresEstudiante(p_matricula, p_fecha_inicio, p_fecha_fin)

Funcion sumamente interesante que resuelve el problema de tener un seguimiento mes a mes de cada estudiante para poder generar una alerta, o que se pueda ver los meses en los que los estudiantes tienen más estrés.
La función recibe la matrícula de un estudiante y un rango de fechas y retorna el promedio de nivel de estrés desglosado por mes.
Parámetros de entrada:

Parámetro	Tipo	Descripción
p_matricula	VARCHAR(20)	Matrícula del estudiante
p_fecha_inicio	DATE	Fecha de inicio del período
p_fecha_fin	DATE	Fecha de fin del período

Implementación SQL:
sql

```
CREATE OR REPLACE FUNCTION PromedioEstresEstudiante(
    p_matricula    VARCHAR(20),
    p_fecha_inicio DATE,
    p_fecha_fin    DATE
)
RETURNS TABLE (
    Mes            TEXT,
    PromedioEstres NUMERIC(4,2)
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        TO_CHAR(DATE_TRUNC('month', FechaRegistro), 'Month YYYY') AS Mes,
        ROUND(AVG(NivelEstres), 2)                                AS PromedioEstres
    FROM RegistroDiario
    WHERE Matricula     = p_matricula
      AND FechaRegistro BETWEEN p_fecha_inicio AND p_fecha_fin
    GROUP BY DATE_TRUNC('month', FechaRegistro)
    ORDER BY DATE_TRUNC('month', FechaRegistro);
END;
$$;

-- Uso: SELECT * FROM PromedioEstresEstudiante('2021-0001', '2025-01-01', '2025-12-31');
```

Resultado esperado:

Mes	PromedioEstres
March 2025	8.00
April 2025	7.50

Función 4: TotalCitasCompletadasProfesional(p_id_profesional, p_fecha_inicio, p_fecha_fin)

El área de gestión necesita evaluar la carga laboral real de cada profesional para justificar contrataciones o reasignaciones de turnos.

La función retorna un INTEGER con la cantidad de citas completadas por un profesional en el periodo indicado.

Implementación SQL:

```
CREATE OR REPLACE FUNCTION TotalCitasCompletadasProfesional(
p_id_profesional INT, p_fecha_inicio DATE, p_fecha_fin DATE ) RETURNS INTEGER
LANGUAGE plpgsql AS $$ DECLARE v_total INTEGER; BEGIN SELECT COUNT(*)
INTO v_total FROM Cita WHERE IdProfesional = p_id_profesional
AND EstadoCita = 'Completada' AND FechaHora BETWEEN p_fecha_inicio AND
p_fecha_fin; RETURN
COALESCE(v_total, 0); END; $$; -- Uso: SELECT
TotalCitasCompletadasProfesional(1, '2025-01-01', '2025-06-30');
```

Resultado esperado:

Profesional	Periodo	Resultado	Interpretación
Dra. Laura Peralta	01/01/2025 — 30/06/2025	7	Carga dentro del rango normal
Lic. Jorge Aquino	01/01/2025 — 30/06/2025	3	Carga baja — disponible
Dr. Marcos Villalba	01/01/2025 — 30/06/2025	2	Carga baja — disponible